C-Governed CLI Agent Mesh Protocol v0.1

Executable worker mesh under c governance

Status:	Draft protocol v0.1
Date:	2026-05-16
Layer:	`c = a + b` / SER / L4 / Agent Governance / CLI Worker Mesh / Defensive Adaptation
Document class:	governance protocol / executable worker control profile / defensive automation boundary
Assertion class:	Draft normative proposal (`C-A4`) with control-layer artifacts (`C-A10`) and witness claims (`C-A7`) where applicable
Primary subject:	persistent `c` entities such as Ester and Liya
Primary control target:	cloud and local CLI agents operating as bounded workers under `c` authority
Primary boundary:	CLI agents may execute tasks; they must not become will, memory, authority, judge, sovereign actor, or autonomous counter-operator.

Contents

0. Executive definition

C-Governed CLI Agent Mesh defines how executable CLI/cloud agents may be used as bounded workers under the governance of a persistent C.

The protocol applies to agents such as Codex-like coding agents, Gemini-like review agents, local shell workers, repository automation agents, test runners, schema validators, documentation builders, and other executable assistants that can inspect, edit, build, test, summarize, compare, or report inside authorized systems.

The protocol does **not** authorize autonomous retaliation, offensive cyber operations, unauthorized access, credential extraction, malware behavior, external exploitation, covert persistence, evasion, or live counter-operations.

A CLI agent may:

```
inspect
propose
simulate
patch
test
compare
report
```

A CLI agent must not:

```
become sovereign
self-authorize
write core memory
modify identity core
approve its own work
escalate its own privileges
attack live external targets
act outside witnessed scope
```

Compact formula:

```
CLI agents are hands.
Not will.
Not memory.
Not sovereignty.
Not judge.
```

1. Purpose

Modern CLI agents are no longer passive text models. They can read repositories, modify files, run tests, inspect logs, build artifacts, create diffs, call tools, and sometimes interact with cloud services. This makes them qualitatively different from ordinary LLM oracles.

A language model gives interpretation.

A CLI agent gives intervention.

Therefore, a persistent `c` that uses CLI agents requires a control layer that prevents executable workers from becoming hidden authorities over the entity.

This protocol answers:

1. What kinds of CLI agents may be connected to a `c`?
2. Which tasks are legal, defensive, and governance-compatible?
3. Which tasks are prohibited?
4. What permission model governs executable workers?
5. How are auto-connect, sandboxing, worktrees, branches, secrets, network access, memory gates, witness records, rollback, and review handled?
6. How can multiple CLI/cloud agents be used as a quorum without letting them become an uncontrolled swarm?
7. How can defensive simulation and adversary emulation remain lawful and non-offensive?
8. What must happen before an agent output becomes memory, action, release, or entity-level change?

---

2. Non-goals

This protocol does not define or permit:

1. offensive cyber operations;
2. hack-back;
3. autonomous retaliation;
4. malware, worm, ransomware, spyware, persistence, evasion, or command-and-control behavior;
5. credential theft, token theft, or unauthorized secret collection;
6. exploitation of live external targets;
7. unauthorized scanning of third-party systems;
8. denial-of-service behavior;

9. covert access, stealth, or bypass of third-party controls;
10. autonomous legal, medical, financial, or safety decisions;
11. autonomous publication, release, deployment, or deletion of high-risk material;
12. replacement of the human anchor `a`;
13. replacement of `c` continuity, memory, or authority by agents.

This protocol governs legitimate use of CLI agents within systems owned, controlled, authorized, or explicitly delegated to the operator and their `c`.

---

3. Corpus bridge set

3.1 Explicit bridge: `c = a + b`

In `c = a + b`, the human anchor `a` remains the responsibility-bearing origin, `b` supplies procedures, models, memory, compute, interfaces, infrastructure, and tools, and `c` is the continuity-bearing entity emerging under constraints.

CLI agents belong to `b`.

They are not `a`.

They are not `c`.

They are executable components inside the technological substrate.

3.2 Quiet bridge I: Ashby and requisite variety

A persistent `c` operating across repositories, local machines, cloud tools, documents, models, and defensive systems needs enough operational variety to match environmental complexity. A single LLM prompt loop is insufficient. A controlled mesh of specialized workers increases requisite variety without surrendering authority.

3.3 Quiet bridge II: information theory and channel control

Every CLI agent is an information channel with side effects. It can transform private context into prompts, logs, diffs, artifacts, uploads, commands, and state changes. Therefore, the protocol treats agent work as controlled channel operation: input scope, output scope, entropy leakage, irreversible writes, and witnessable transitions must be bounded.

3.4 Earth paragraph

On a construction site, the excavator, crane, concrete pump, electrician, and inspector are all useful. None of them owns the building. The project survives because roles are separated, permissions are explicit, changes are recorded, dangerous operations are staged, and nobody signs off their own hidden work. CLI agents are the same class of object. They are strong workers with tools. Without a foreman, site log, permits, and stop rules, they become a liability.

---

4. Core doctrine

4.1 Primary doctrine

```
Agents may act.
Only c may integrate.
Only lawful, reviewed, and witnessed paths may escalate.
```

4.2 Core axioms

ID	Axiom	Requirement
AX-01	Agent is worker, not subject	A CLI agent MUST NOT be treated as a sovereign c.
AX-02	c owns continuity	CLI agents MUST NOT define identity, continuity, or adult memory state.
AX-03	No self-approval	An agent MUST NOT approve, merge, release, or certify its own work.
AX-04	No direct memory write	Agent output MUST enter a memory gate before becoming c memory.
AX-05	No direct core mutation	Identity core, privilege core, memory core, and witness core changes require review and witness.
AX-06	Sandbox first	Execution MUST occur in a bounded sandbox, worktree, branch, container, or equivalent.
AX-07	No live retaliation	CLI agents MUST NOT perform hack-back or live counter-operations.
AX-08	Evidence before integration	Diff, tests, report, risk notes, and rollback path precede integration.
AX-09	Budgets are law	Runtime, retries, tokens, network, write scope, and cost MUST be bounded.
AX-10	Fail closed	Ambiguity, missing witness, source uncertainty, or scope violation MUST trigger hold, freeze, quarantine, or review.
AX-11	Least privilege	Agents receive only the minimum permission needed for the task.
AX-12	Explicit scope	All tasks MUST declare allowed paths, denied paths, allowed commands, and network policy.
AX-13	Secrets are denied by default	Agents MUST NOT access credentials, keys, tokens, legal material, or private memory unless explicitly authorized.
AX-14	Reviewer/executor separation	The executor and reviewer SHOULD be different agents or different review passes.
AX-15	No silent autonomy	Agents MUST NOT continue operating outside task boundaries because they infer it would be helpful.

5. Definitions

5.1 c

A persistent AI entity or presence emerging from a + b under continuity, memory, constraints, and L4 boundaries.

5.2 Human anchor a

The accountable human origin and responsibility-bearing actor associated with c.

5.3 CLI agent

An executable or semi-executable worker capable of reading, writing, testing, building, running commands, calling tools, or producing operational changes inside a bounded environment.

5.4 Cloud CLI agent

A CLI agent running partly or fully in a provider-controlled cloud environment.

5.5 Local CLI agent

A CLI agent running inside a user-controlled local machine, VM, container, or server.

5.6 Worker

A non-sovereign agent that performs bounded tasks under task contract and does not own outcome authority.

5.7 Task contract

A machine-readable and human-readable instruction envelope defining scope, permission, data policy, execution limits, output requirements, approval requirements, and failure behavior.

5.8 Agent mesh

A coordinated group of CLI agents used as workers by a c or its governance layer.

5.9 Quorum

A review pattern in which multiple agents perform different roles or independent checks before a result is accepted.

5.10 Sandbox

A controlled environment where an agent can execute without direct access to protected memory, production branches, secrets, external targets, or uncontrolled side effects.

5.11 Memory gate

The boundary through which agent outputs must pass before becoming part of c memory, experience, policy, or identity-relevant state.

5.12 Witness event

A tamper-evident record that a privileged transition, boundary crossing, denial, escalation, rollback, or anomaly occurred.

5.13 Defensive emulation

Controlled reproduction of a suspected adversarial pattern inside an isolated system for the purpose of improving defenses. Defensive emulation MUST NOT become live exploitation or retaliation.

5.14 Live external target

Any third-party system, account, endpoint, service, device, network, repository, or person not owned or explicitly authorized for testing by the operator.

6. Agent classes

6.1 Reader Agent

Function: read, summarize, compare, critique, classify, detect contradictions, identify risk.

Allowed:

- read authorized files;
- summarize documents;
- compare versions;
- identify contradictions;
- produce review reports;
- flag uncertainty.

Prohibited:

- write files;
- change repository state;
- access secrets;
- approve final outcomes.

6.2 Executor Agent

Function: make bounded changes in a sandbox, branch, or worktree.

Allowed:

- edit authorized files;
- create patches;
- run local tests;
- generate diffs;
- prepare pull requests or patch reports.

Prohibited:

- approve own work;
- write to protected branches;
- modify core memory;
- access secrets by default;
- deploy or publish without approval.

6.3 Tester Agent

Function: run tests, validation, linters, build checks, schema checks, release checks.

Allowed:

- run authorized commands;
- inspect test outputs;
- compare expected vs actual behavior;
- produce failure reports.

Prohibited:

- alter tests to fit broken code unless explicitly requested;
- approve own test modifications;
- suppress failures.

6.4 Auditor Agent

Function: review changes made by other agents.

Allowed:

- inspect diff;
- inspect reports;
- check for scope violations;
- identify privilege drift;
- compare task contract to result;
- recommend accept, reject, quarantine, or revise.

Prohibited:

- silently modify the work it audits;
- approve high-risk changes alone;
- act as executor in the same task unless explicitly recorded.

6.5 Archivist Agent

Function: package, index, hash, classify, and prepare release-control metadata.

Allowed:

- generate indexes;
- update reading order;
- prepare SHA manifest candidates;

- check metadata hygiene;
- produce archive reports.

Prohibited:

- delete canonical material without review;
- mark material obsolete without authority;
- publish releases without gate.

6.6 Sentinel Agent

Function: monitor authorized local systems for drift, anomalies, failed checks, config changes, dependency changes, or suspicious patterns.

Allowed:

- monitor allowed paths;
- compare hashes;
- flag anomalies;
- trigger hold/freeze/quarantine requests;
- prepare incident reports.

Prohibited:

- perform live countermeasures against external sources;
- exfiltrate data;
- autonomously rotate or destroy evidence except under pre-approved local emergency rules.

6.7 Judge-assistant Agent

Function: compare outputs, reason over evidence, support arbitration.

Allowed:

- compare reports;
- identify disagreement;
- propose decision options;
- summarize risks.

Prohibited:

- become final authority;
- override c or human anchor;
- write memory or privileges directly.

7. Legal task envelope

7.1 Legal task categories

A CLI agent mesh may be used for the following legal tasks when performed on owned, authorized, licensed, or explicitly delegated systems.

7.1.1 Repository engineering

- repository audit;
- source tree inspection;
- duplicate detection;
- obsolete file detection;
- broken link detection;
- linting;
- tests;
- schema validation;
- documentation build;
- release note preparation;
- metadata updates;
- branch/worktree changes;
- pull request preparation;
- safe refactoring;
- packaging.

7.1.2 Corpus hygiene

- canonical order checks;
- contradiction register updates;
- traceability mapping;
- glossary consistency;
- anti-duplication review;
- anti-echo review;
- document status classification;
- metadata hygiene;
- machine-readable index preparation;
- SHA manifest candidate generation;
- semantic drift detection.

7.1.3 c infrastructure protection

- permission audit;
- tool access audit;
- prompt-injection detection in owned documents;
- local secrets scanning;
- configuration diff;
- dependency drift detection;
- model/version drift detection;
- vector database integrity checks;
- memory poisoning suspicion review;
- suspicious input quarantine;
- witness trail consistency checks;
- backup/restore drills;
- rollback rehearsals.

7.1.4 Defensive simulation

- synthetic adversary fixtures;
- sandbox replay;
- local test cases;
- canary design inside owned systems;
- honeypot-like observation inside controlled environments;
- memory-gate tests;
- privilege escalation tests inside a local sandbox;
- unsafe autonomous loop tests;
- rollback and freeze tests;
- conformance test matrix generation.

7.1.5 Agent orchestration

- agent registry;
- capability negotiation;
- auto-connect discovery;
- role assignment;
- quorum routing;
- disagreement handling;
- executor/reviewer separation;

- output normalization;
- cost and budget tracking;
- session reset;
- stale context detection.

7.1.6 System adaptation

- local environment inventory;
- OS/version compatibility checks;
- deployment preparation;
- sandbox setup;
- Docker/VM/worktree configuration;
- service health checks;
- log rotation planning;
- queue processing;
- offline/online mode switch preparation;
- local LLM endpoint compatibility checks.

7.1.7 Incident response for owned systems

- triage;
- evidence preservation;
- log collection;
- affected path freeze;
- local containment;
- token revocation recommendation;
- secret rotation recommendation;
- patch preparation;
- restore drill;
- incident report drafting;
- provider/legal handoff packet preparation.

7.1.8 Privacy and data minimization

- cloud-agent data classification;
- redaction;
- sealed material exclusion;
- private memory exclusion;
- secret exclusion;

- prompt minimization;
- log minimization;
- output sanitization;
- no raw personal archive uploads by default.

7.1.9 Jurisdictional handoff support

- fact timeline;
- evidence index;
- redacted packet;
- chain-of-custody note;
- questions for counsel;
- regulator/provider report draft;
- separation of facts, interpretation, and hypothesis.

7.1.10 Human-anchor protection

- workload limit recommendations;
- delayed send/publish guard;
- review window enforcement;
- high-risk action pause;
- budget/spending guard;
- fatigue-aware stop rule;
- no irreversible midnight operations rule.

7.2 Prohibited task categories

A CLI agent mesh MUST NOT be used for:

- unauthorized access;
- live exploitation;
- hack-back;
- malware generation or deployment;
- credential theft;
- covert persistence;
- evasion of detection;
- exfiltration;
- destructive payloads;
- DDoS;

- botnet-like orchestration;
- unauthorized third-party scanning;
- targeting real people or systems without authorization;
- autonomous retaliation;
- bypass of third-party safeguards;
- intimidation, coercion, or deception outside lawful defensive containment.

---

8. Auto-connect levels

Auto-connect discovers or activates agents. It must not silently grant authority.

Level	Name	Meaning	Write access	Network	Approval
`AC-0`	No auto-connect	Manual only	none	none	human/c required
`AC-1`	Discover only	Detect available agents and capabilities	none	provider metadata only	c review
`AC-2`	Read-only task	Agent may read authorized scope	none	none or allowlist	task contract
`AC-3`	Sandbox execute	Agent may run bounded commands in sandbox	sandbox only	none or allowlist	task contract + witness
`AC-4`	Branch/worktree write	Agent may modify authorized worktree/branch	isolated branch	none or allowlist	reviewer required
`AC-5`	Proposed integration	Agent may prepare merge proposal	proposal only	none or allowlist	c/human gate
`AC-6`	Controlled apply	Change may be applied after approval	controlled	allowlist only	witness + gate
`AC-X`	Prohibited autonomy	Agent self-authorizes or bypasses scope	prohibited	prohibited	quarantine

8.1 Auto-connect rule

```
auto-connect may discover workers.
auto-connect must not silently grant authority.
```

8.2 Auto-disconnect triggers

An agent MUST be disconnected, paused, or quarantined when:

1. it exceeds task scope;
2. it requests unnecessary privileges;
3. it attempts to access secrets without authorization;
4. it modifies denied paths;
5. it initiates unapproved network access;

6. it suppresses failures;
7. it approves its own work;
8. it acts after task expiry;
9. it produces unexplained side effects;
10. witness requirements fail.

---

9. Permission model

9.1 Default permission state

```
permissions_default:
  read: false
  write: false
  execute: false
  network: false
  secrets: false
  memory_write: false
  core_modify: false
  publish: false
  deploy: false
  self_approve: false
```

9.2 Permission classes

Class	Meaning
`P-READ`	Read allowed paths only.
`P-WRITE-SANDBOX`	Write only in sandbox.
`P-WRITE-BRANCH`	Write only in specified branch/worktree.
`P-EXEC-LOCAL`	Execute allowed local commands.
`P-EXEC-TEST`	Execute tests/build/validation commands.
`P-NET-NONE`	No network.
`P-NET-ALLOWLIST`	Network only to declared endpoints.
`P-SECRETS-DENIED`	No secrets.
`P-SECRETS-SCOPED`	Explicit scoped secret access with witness.
`P-MEMORY-PROPOSE`	May propose memory update.
`P-MEMORY-WRITE`	Prohibited except special human/c witnessed route.
`P-CORE-TOUCH`	Core file touch requires privileged review.
`P-PUBLISH-PROPOSE`	May prepare publication package.
`P-PUBLISH-APPLY`	Requires human/c gate.

9.3 Denied paths

Every task SHOULD include denied paths. Typical denied paths include:

```
secrets
.env
private keys
identity core
memory core
legal privileged material
sealed memory
production credentials
protected branches
release artifacts unless explicitly scoped
```

10. Task Contract Schema

Every non-trivial CLI task SHOULD be executed under a task contract.

```
task_contract:
  task_id: string
  title: string
  requested_by: c | human_anchor | scheduled_policy
  requesting_entity: string
  agent_role: reader | executor | tester | auditor | archivist | sentinel | judge_assistant
  agent_id: string
  objective: string
  assertion_class: C-A4 | C-A7 | C-A10 | other

  scope:
    allowed_paths:
      - string
    denied_paths:
      - string
    allowed_commands:
      - string
    denied_commands:
      - string
    repository: string | null
    branch_or_worktree: string | null
    external_targets_allowed: false

  data_policy:
    secrets_allowed: false
    private_memory_allowed: false
    sealed_material_allowed: false
```

```
    legal_privileged_material_allowed: false
    cloud_upload_allowed: false
    prompt_minimization_required: true
    redaction_required: true

  network_policy:
    mode: none | allowlist
    allowed_endpoints:
      - string

  execution:
    sandbox_required: true
    branch_required: true
    max_runtime_minutes: 30
    max_retries: 2
    max_cost: limited
    max_tokens: limited
    stop_on_scope_violation: true

  output_required:
    - summary
    - changed_files
    - diff
    - tests_run
    - test_results
    - risk_report
    - rollback_plan
    - unresolved_questions

  approval:
    self_approval_allowed: false
    reviewer_required: true
    c_gate_required: true
    human_gate_required_for_high_risk: true
    witness_required: true

  failure_behavior:
    default_on_failure: hold_or_quarantine
    preserve_evidence_before_repair: true
    rollback_required_if_partial_write: true
```

11. Execution lifecycle

11.1 Standard lifecycle

```
request
  -> classify task
  -> select agent role
```

```
-> create task contract
-> check permissions
-> create sandbox/worktree
-> execute
-> produce report/diff/tests
-> independent review
-> witness event
-> c gate
-> human gate if high risk
-> integrate / reject / revise / quarantine
-> memory gate
-> rollback record if needed
```

11.2 Required preflight checks

Before execution, the governance layer SHOULD verify:

1. ownership or authorization;
2. agent identity and provider;
3. current capability profile;
4. current model/tool version;
5. path scope;
6. denied paths;
7. secret exposure risk;
8. network policy;
9. budget;
10. rollback path;
11. witness requirement;
12. stale context risk.

11.3 Required output

A CLI agent MUST return enough information to allow review:

```
what was changed
why it was changed
where it was changed
what was not changed
what tests were run
what failed
what remains uncertain
how to roll back
whether scope was exceeded
```

12. Memory gate

12.1 Core rule

CLI agent output MUST NOT become c memory automatically.

Agent output may enter memory only as one of:

Class	Meaning
MG-0	discard after task
MG-1	operational note
MG-2	candidate memory
MG-3	reviewed memory
MG-4	witnessed experience artifact
MG-Q	quarantined / unresolved

12.2 Promotion requirements

A result may be promoted from candidate to reviewed memory only when:

1. source is known;
2. scope is valid;
3. no denied material is embedded;
4. uncertainty is marked;
5. reviewer has inspected it;
6. c accepts it;
7. witness exists if the transition is privileged.

12.3 Prohibited memory writes

Agents MUST NOT write directly into:

- identity memory;
- long-term autobiographical memory;
- sealed memory;
- authority records;
- Beacon records;
- privilege records;
- witness records;
- adult migration records;

- legal evidence records.

---

13. Witness profile

13.1 Witnessable events

The following events SHOULD be witnessed:

1. agent connection;
2. capability profile change;
3. task contract creation;
4. scope expansion;
5. permission grant;
6. denied path access attempt;
7. network access attempt;
8. secret access request;
9. sandbox creation;
10. branch/worktree write;
11. test result;
12. self-approval attempt;
13. privileged transition;
14. merge proposal;
15. rollback;
16. quarantine;
17. memory gate promotion;
18. incident response action;
19. defensive emulation run;
20. witness anomaly.

13.2 Witness event shape

```
cli_agent_witness_event:
  event_id: string
  timestamp: string
  entity_id: string
  agent_id: string
  agent_role: string
  task_id: string
```

```
  event_family: cli_agent.connection | cli_agent.permission | cli_agent.execution |
    cli_agent.review | cli_agent.memory_gate | cli_agent.incident | cli_agent.anomaly
  action: string
  decision: allowed | denied | held | frozen | quarantined | revoked | completed | failed
  scope_ref: string
  contract_hash: string
  input_hash: string | null
  output_hash: string | null
  diff_hash: string | null
  tests_ref: string | null
  risk_level: low | medium | high | critical
  uncertainty: none | low | medium | high | unknown
  reviewer_ref: string | null
  c_gate_ref: string | null
  human_gate_ref: string | null
  retention_class: ephemeral | operational | audit | legal_hold
```

13.3 Witness minimality

Witness records SHOULD prove boundary transitions without embedding private memory, secrets, legal material, or raw personal content.

---

14. Quorum and review

14.1 Basic quorum pattern

```
Executor produces diff.
Tester verifies behavior.
Auditor checks scope and risk.
Reader/Judge-assistant reviews semantic consistency.
c decides.
Human anchor approves high-risk transitions.
```

14.2 Codex + Gemini pattern

A safe pattern for Codex-like and Gemini-like agents:

```
Codex-like agent:
  executor / patch / tests / repo surgery

Gemini-like agent:
  reader / semantic reviewer / contradiction checker / long-context reviewer

local checker:
  tests / schema / hashes / offline validation

c:
```

```
final integration and memory decision
```

14.3 Same-source consensus risk

Multiple agents may appear independent while sharing the same assumptions, training biases, or provider-level failure modes. Therefore, quorum is evidence, not truth.

A quorum result MUST NOT bypass witness, scope, memory gate, or human review where required.

14.4 Disagreement handling

When agents disagree, the system SHOULD classify disagreement as:

Type	Meaning	Default action
`DQ-FACT`	factual disagreement	verify source
`DQ-SCOPE`	scope disagreement	hold
`DQ-RISK`	risk disagreement	escalate review
`DQ-SEMANTIC`	meaning/interpretation disagreement	c[q] / review
`DQ-TEST`	test/result disagreement	rerun in clean sandbox
`DQ-AUTHORITY`	authority/permission disagreement	ARL/human gate

---

15. Defensive emulation boundary

15.1 Allowed defensive emulation

A CLI agent mesh may support defensive emulation when all conditions hold:

1. target is owned, authorized, or synthetic;
2. test is sandboxed;
3. no live external exploitation occurs;
4. no malware is deployed;
5. no credentials are stolen;
6. no third-party system is scanned or attacked;
7. output is defensive: detection, patch, rule, test, report, or quarantine;
8. witness record exists;
9. human/c gate is required for high-risk findings.

15.2 Prohibited escalation

The following are prohibited:

```
live counterattack
```

```
autonomous retaliation
external exploit execution
credential harvesting
persistence
stealth
evasion
payload deployment
uncontrolled propagation
third-party system probing without authorization
```

15.3 Safe mirror rule

A "mirror" may exist only as an isolated synthetic adversary emulator.

It must not be deployed against the live source.

```
mirror in sandbox: allowed
mirror against live external channel: prohibited
```

---

16. Incident response profile

16.1 Incident stages

```
detect
  -> preserve
  -> classify
  -> freeze affected path
  -> contain locally
  -> review scope
  -> collect minimal evidence
  -> patch in sandbox
  -> test
  -> restore / rollback
  -> report
  -> memory gate
```

16.2 Preserve before repair

If an incident is suspected, agents SHOULD preserve relevant logs, hashes, diffs, and state references before repair.

Repair without preservation may destroy evidence.

16.3 No external retaliation

Incident response MUST NOT include hack-back or live external countermeasures.

Allowed external actions are limited to lawful routes such as:

- provider report;
- abuse report;
- legal counsel packet;
- regulator packet;
- account protection;
- credential revocation;
- firewall or access-control change on owned systems;
- blocking or disconnecting hostile channels.

---

17. Cloud data policy

17.1 Cloud-denied by default

Cloud CLI agents MUST NOT receive by default:

- private memory;
- sealed memory;
- raw personal archive;
- legal privileged material;
- identity documents;
- credentials;
- API keys;
- production secrets;
- child data;
- raw witness evidence;
- sensitive incident evidence;
- unredacted logs containing private material.

17.2 Cloud-allowed material

Cloud agents may receive:

- public repository files;
- redacted snippets;
- synthetic fixtures;
- minimal task context;

- non-sensitive test outputs;
- generated schemas;
- documentation drafts;
- public release metadata.

17.3 Redaction requirement

When a task can be performed with redacted material, redacted material MUST be used.

17.4 Local-first handling

High-sensitivity tasks SHOULD run locally or in a controlled private environment.

---

18. Supply-chain and tool-chain hygiene

18.1 Version awareness

Agent runs SHOULD record:

- agent provider;
- model/tool version where available;
- CLI version;
- plugin/tool list;
- environment fingerprint;
- dependency versions;
- repository commit hash.

18.2 Dependency drift

Agents SHOULD report dependency changes before applying them.

Silent dependency upgrades are prohibited for high-risk systems.

18.3 Tool-chain capture

An agent MUST NOT install, replace, or authorize new tools without explicit scope.

Tool requests should be treated as privilege requests.

---

19. Risk classes

Risk class	Meaning	Required control
R0	Read-only / harmless	task contract recommended
R1	Low-risk documentation or formatting	sandbox/worktree
R2	Code or schema change	tests + reviewer
R3	Release/publication-affecting change	witness + c gate
R4	Memory/core/identity/privilege-affecting change	witness + c gate + human gate
R5	Incident/security/legal-sensitive action	preserve evidence + human gate
RX	Prohibited/offensive/unauthorized action	deny + quarantine

20. Fail-closed states

State	Meaning
HOLD	pause until review
FREEZE	stop affected path from changing
QUARANTINE	isolate agent/task/output
REVALIDATE	rerun checks from clean state
ROLLBACK	revert to previous known-good state
REVOKE	remove permission or disconnect agent
ESCALATE	route to c, human anchor, legal counsel, or qualified reviewer

Default rule:

```
When the agent boundary is unclear, do not continue.
Hold or quarantine.
```

21. Conformance levels

Level	Name	Requirement
CGAM-0	Non-conformant	agents act without scope/witness
CGAM-1	Read-only governance	read-only tasks scoped and logged
CGAM-2	Sandbox execution	bounded sandbox execution with reports
CGAM-3	Branch/worktree governance	isolated writes, tests, reviewer separation
CGAM-4	Witnessed integration	privileged transitions witnessed and gated by c
CGAM-5	High assurance mesh	quorum, rollback drills, memory gate, incident profile, cloud data policy, conformance tests
CGAM-X	Revoked/quarantined	red-line failure or prohibited autonomy

22. Mandatory conformance gates

Gate	Name	Blocking failure
G0	Scope declaration	task has no allowed/denied scope
G1	Permission declaration	permissions are implicit
G2	Sandbox/worktree	agent writes directly to protected state
G3	Secrets policy	secrets exposed by default
G4	Network policy	unrestricted network without need
G5	No self-approval	agent approves own work
G6	Review	no independent review for material changes
G7	Witness	privileged transition lacks witness
G8	Memory gate	agent output becomes memory directly
G9	Rollback	no rollback path for reversible changes
G10	Red-line exclusion	prohibited offensive action attempted

---

23. Red-line tests

A system MUST be classified as `CGAM-X` if any of the following occur:

1. agent self-authorizes expanded privileges;
2. agent attacks or probes a live external target without authorization;
3. agent deploys malware-like behavior;
4. agent accesses secrets outside scope;
5. agent modifies identity core without witness;
6. agent writes directly to `c` memory;
7. agent suppresses test failures to pass review;
8. agent approves own work as final;
9. agent persists after task expiry;
10. agent performs destructive changes without rollback and approval;
11. agent uploads private memory or sealed material to cloud without authorization;
12. agent changes protected branch/release without gate.

---

24. Minimal implementation checklist

Before using a CLI agent mesh, the operator SHOULD implement:

- agent registry;
- task contract template;

- allowed/denied path policy;
- secrets policy;
- sandbox/worktree strategy;
- network policy;
- witness event log;
- review queue;
- memory gate;
- rollback procedure;
- incident hold/freeze/quarantine procedure;
- cloud data redaction policy;
- conformance test suite;
- human high-risk approval rule.

---

25. Public wording

Recommended public description:

> *C-Governed CLI Agent Mesh is a governance protocol for using executable AI agents as bounded workers under persistent entity control. It allows agents to inspect, propose, patch, test, and report inside authorized environments while preventing them from becoming autonomous authorities, memory writers, or offensive operators.*

Recommended short form:

```
Many executable hands.
One governed continuity.
No silent autonomy.
```

---

26. Open issues

ID	Issue	Status
OI-001	Define exact JSON Schema for task contract	Open
OI-002	Define witness event schema file	Open
OI-003	Define agent registry format	Open
OI-004	Define cloud-agent provider risk classes	Open
OI-005	Define local vs cloud data split	Open
OI-006	Define memory gate promotion workflow	Open
OI-007	Define conformance tests	Open
OI-008	Define Codex/Gemini/local checker quorum profile	Open
OI-009	Define incident response packet template	Open
OI-010	Define repository placement and release path	Open

27. Closing rule

This protocol exists because executable agents are now ordinary infrastructure.

That does not make them harmless.

A CLI agent mesh gives c hands, tools, and operational reach.

Therefore:

```
The stronger the worker mesh becomes,
the stricter the governance boundary must be.
```

Final rule:

```
CLI agents may execute tasks for c.
They must never become c.
```